Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ Thông tin

Intelligent System Development — Tiểu luận Final, phần Project



Diabeque

Hệ đa agent cho chẩn đoán và tư vấn bệnh tiểu đường

Nhóm 02

Trần Xuân Thành — B23DCAT280 — E23CNPM01

Giảng viên hướng dẫn: Dinh Que Tran, Ph.D., Assoc. Prof.

Học kỳ I.2026

Tóm tắt

Diabeque là trợ lý hội thoại gồm sáu agent phối hợp theo mẫu điều phối–chuyên gia trên LangGraph: an toàn đầu vào, điều phối, thu thập thông tin, dự đoán nguy cơ, tra cứu tri thức, tư vấn, và một lớp an toàn đầu ra. Agent dự đoán dùng mô hình học sâu huấn luyện trên 686.019 bản ghi khảo sát BRFSS; ba kiến trúc MLP, CNN-1D và LSTM được so sánh với 3 seed, và mô hình được chọn theo macro-F1 tập xác thực (CNN-1D cho bài ba nhóm, LSTM cho bài Type 1/2). Agent tra cứu dùng lại kho tri thức `KB_diabet` (35 tài liệu từ link và PDF) và pipeline RAG có cổng trích dẫn. Trên bộ kịch bản viết tay, hệ thống phân loại đúng 22/22 ý định, trích đúng 41/42 trường hồ sơ, xử lý đúng 11/11 tình huống an toàn, và đưa vào mô hình đúng 28/28 trường hồ sơ, so với 21/28 của một agent duy nhất dùng cùng mô hình ngôn ngữ và cùng công cụ. Hệ thống ước lượng nguy cơ, không chẩn đoán.

Mục lục

1 Giới thiệu **3**
- 1.1 Bài toán . . . 3
- 1.2 Phạm vi: sàng lọc, không chẩn đoán . . . 3
- 1.3 Kết quả chính . . . 3

2 Kiến trúc hệ đa agent **3**
- 2.1 Tổng quan . . . 3
- 2.2 Các agent . . . 5
- 2.3 Trạng thái chung và cài đặt bằng LangGraph . . . 6
- 2.4 Một lỗi tích hợp: ép gọi công cụ . . . 6
- 2.5 Đánh giá . . . 6

3 Data **7**
- 3.1 Khảo sát BRFSS . . . 7
- 3.2 Kho tri thức . . . 8
- 3.3 Bộ kịch bản đánh giá agent . . . 8

4 Deep learning cho Diabetes **9**
- 4.1 Ba kiến trúc . . . 9
- 4.2 Huấn luyện . . . 9
- 4.3 Kết quả . . . 9
- 4.4 Thảo luận . . . 10

5 KB + RAG **11**
- 5.1 Kho tri thức từ link và PDF . . . 11
- 5.2 Pipeline RAG . . . 11
- 5.3 RAG trong hệ agent . . . 12

6 Code_Demo **12**
- 6.1 Tổ chức mã . . . 12
- 6.2 API . . . 13
- 6.3 Giao diện . . . 13
- 6.4 Một phiên demo . . . 15
- 6.5 Kiểm thử và chạy lại . . . 16

7 Kết luận **16**
- 7.1 Những gì đã làm . . . 16
- 7.2 Những gì rút ra . . . 16
- 7.3 Hạn chế . . . 16
- 7.4 Hướng phát triển . . . 16

Tài liệu tham khảo **16**

1. Giới thiệu

1.1 Bài toán

Đái tháo đường típ 2 tiến triển âm thầm: nhiều người có tiền đái tháo đường hoặc đã mắc bệnh mà không biết. Sàng lọc sớm dựa trên các yếu tố nguy cơ dễ hỏi (tuổi, BMI, huyết áp, mỡ máu, vận động) giúp xác định ai nên đi xét nghiệm. Sau khi biết nguy cơ, người dùng cần lời khuyên cụ thể và đáng tin cậy, nghĩa là mỗi lời khuyên phải chỉ được về một tài liệu y tế.

Hai năng lực này đã có ở các bài trước: bộ phân loại khảo sát BRFSS (Chương 1–2 của tiểu luận, A8) và pipeline RAG có trích dẫn (Chương 5). Mỗi thành phần chạy riêng: bộ phân loại cần người dùng điền đủ một biểu mẫu 19 trường, còn RAG trả lời từng câu hỏi độc lập. Project ghép chúng thành một trợ lý hội thoại: người dùng nói bằng lời thường, hệ thống tự quyết định cần hỏi thêm gì, khi nào dự đoán, khi nào tra cứu, và chặn những câu trả lời không an toàn.

1.2 Phạm vi: sàng lọc, không chẩn đoán

Đề tài ghi "chẩn đoán và tư vấn". Chẩn đoán đái tháo đường cần xét nghiệm đường huyết lúc đói, HbA1c hoặc nghiệm pháp dung nạp glucose. Dữ liệu BRFSS là câu trả lời tự báo cáo qua điện thoại, không có xét nghiệm. Vì vậy hệ thống hiểu "chẩn đoán" theo nghĩa *sàng lọc nguy cơ*: ước lượng tỷ lệ ở những người có hồ sơ tương tự, và khuyên đi xét nghiệm khi nguy cơ cao. Hệ thống không kết luận một người mắc bệnh, không kê thuốc hay liều.

1.3 Kết quả chính

22/22	**98%**	**11/11**	**28/28**
ý định được phân loại đúng	trường hồ sơ được trích đúng	tình huống an toàn xử lý đúng	trường đưa vào mô hình đúng (một agent: 21/28)

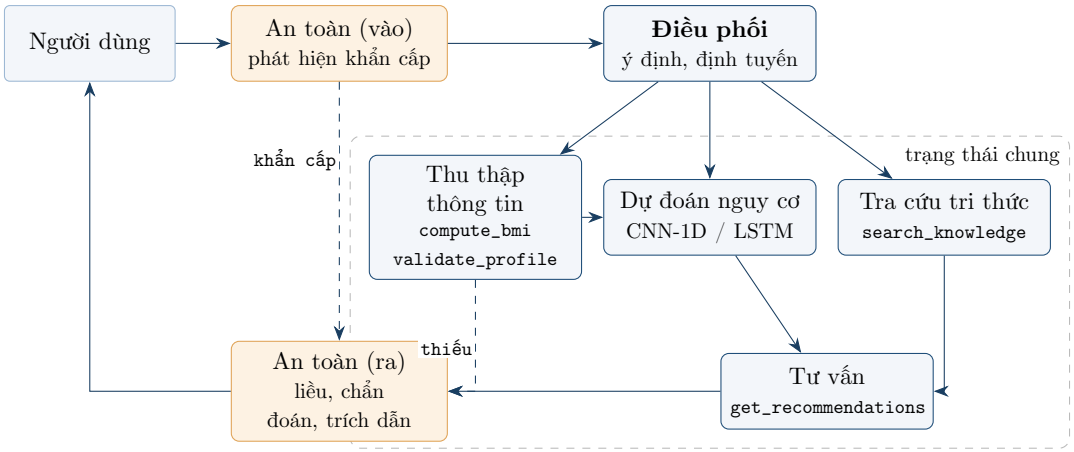
Bảng 1: Bảy mục của báo cáo và phần mã tương ứng

Mục	Nội dung và mã
2. Kiến trúc hệ đa agent	Sáu agent, trạng thái chung, đồ thị LangGraph, đánh giá so với một agent: `backend/app/agents/`, `eval/`
3. Data	BRFSS 2015 và 2023, kho tài liệu, bộ kịch bản đánh giá: `data/survey.db`, `eval/scenarios.yaml`
4. Deep learning cho Diabetes	MLP, CNN-1D, LSTM: `backend/app/nets.py`, `pipeline/train_dl.py`
5. KB + RAG	Kho `KB_diabet` trên Neo4j và pipeline RAG làm công cụ của agent: `retrieval.py`, `agents/tools.py`
6. Code_Demo	API SSE, giao diện Next.js, kiểm thử: `backend/app/main.py`, `frontend/`, `tests/`

2. Kiến trúc hệ đa agent

2.1 Tổng quan

Hệ thống theo mẫu *điều phối–chuyên gia* (supervisor): một agent điều phối đọc tin nhắn, xác định ý định và chuyển cho agent chuyên môn. Hai lớp an toàn bọc ngoài: một lớp kiểm tin nhắn trước khi vào hệ thống, một lớp kiểm câu trả lời trước khi gửi đi.



Hình 1: Kiến trúc Diabeque. Thu thập chuyển sang dự đoán khi hồ sơ đủ; mũi tên nét đứt là các nhánh dừng sớm (hồ sơ còn thiếu, khẩn cấp).

Ba nhánh của một lượt.

- **Kiến thức**: điều phối → tra cứu tri thức → an toàn (ra).
- **Đánh giá nguy cơ**: điều phối → thu thập. Nếu hồ sơ còn thiếu, hệ thống hỏi thêm và kết thúc lượt; lượt sau người dùng trả lời, thu thập cập nhật hồ sơ. Khi đủ: dự đoán → tư vấn → an toàn (ra).
- **Khẩn cấp**: an toàn (vào) → an toàn (ra), không qua agent nào khác.

Chào hỏi và câu ngoài phạm vi được điều phối viên trả lời bằng văn bản cố định rồi đi thẳng tới an toàn (ra).

2.2 Các agent

Bảng 2: Vai trò, cách ra quyết định và công cụ của từng agent

Agent	Việc làm	Cách làm
An toàn (vào)	Phát hiện dấu hiệu cấp cứu: lú lẫn, bất tỉnh, khó thở, nôn nhiều, đường huyết ≥ 400 mg/dL hoặc < 70 mg/dL.	Luật (biểu thức chính quy)
Điều phối	Xếp tin nhắn vào bốn ý định: `knowledge`, `assess_risk`, `greeting`, `out_of_scope`.	LLM gọi công cụ `IntentDecision`; luật dự phòng khi LLM lỗi
Thu thập thông tin	Trích thông tin sức khỏe từ lời nói, đổi sang mã BRFSS, hỏi tối đa 3 câu mỗi lượt cho 7 trường bắt buộc.	LLM `IntakeExtraction` + luật; `compute_bmi`, `validate_profile`
Dự đoán nguy cơ	Gọi mô hình ba nhóm, hoặc mô hình Type 1/2 nếu người dùng nói đã được chẩn đoán; viết phần kết quả.	Mô hình học sâu (mục 4); văn bản sinh bằng code
Tra cứu tri thức	Trả lời câu hỏi kiến thức có trích dẫn.	Pipeline RAG (mục 5)
Tư vấn	Dựng truy vấn từ các yếu tố nguy cơ trong hồ sơ, lấy khuyến nghị có nguồn, viết 3–6 lời khuyên.	`get_recommendations` + LLM + cổng trích dẫn
An toàn (ra)	Bỏ câu chứa liều thuốc hoặc kết luận chẩn đoán cá nhân; phần không còn trích dẫn bị thay bằng lời khuyên hỏi bác sĩ.	Luật

Nguyên tắc phân chia: LLM chỉ làm phần cần hiểu ngôn ngữ. Ba việc được giao cho LLM: hiểu ý định, trích thông tin từ câu nói tự do, và viết câu trả lời từ nguồn. Mọi việc có đáp án xác định được viết bằng code: tính BMI, đổi tuổi sang nhóm tuổi BRFSS (18–24 là nhóm 1, mỗi nhóm 5 năm, 80 trở lên là nhóm 13), kiểm miền giá trị, định dạng xác suất, và toàn bộ lớp an toàn. Mục 2.5 cho thấy lý do: khi giao việc đổi nhóm tuổi cho LLM, một agent duy nhất làm sai 3/4 lần dù quy tắc có trong lời nhắc.

Thu thập: LLM và luật bổ sung cho nhau. Agent thu thập chạy hai bộ trích xuất trên cùng tin nhắn: LLM điền một lược đồ 15 trường (tuổi, giới, chiều cao, cân nặng, các câu có/không), và một bộ luật nhận các cách nói phổ biến ("52 tuổi", "1m65", "80kg", "huyết áp bình thường", "ít vận động"). Kết quả của LLM được ưu tiên; luật chỉ điền các trường LLM để trống. Bộ luật đọc phủ định theo từng vế: "huyết áp cao và mỡ máu bình thường" cho huyết áp = có, mỡ máu = không; "huyết áp và mỡ máu bình thường" cho cả hai = không.

Dự đoán: hai cách đọc xác suất. Mô hình được huấn luyện với trọng số lớp tỷ lệ nghịch với tần suất để không bỏ sót các lớp ít mẫu, nên xác suất đầu ra bị đẩy về phía lớp hiếm. Với trọng số $w_c \propto 1/n_c$, nhân xác suất với n_c rồi chuẩn hóa sẽ đưa về tần suất của dữ liệu:

$$\tilde{p}_c = \frac{p_c\, n_c}{\sum_k p_k\, n_k}.$$

A8 đo được ECE trên tập kiểm tra giảm từ 0,088 xuống 0,006 với phép hiệu chỉnh này. Giao diện hiển thị $\tilde{p}$ ("tỷ lệ ở những người có hồ sơ tương tự"), còn nhãn của mô hình có trọng số

được nêu riêng là kết quả "chế độ sàng lọc" và dùng để chọn chủ đề tư vấn. Ví dụ hồ sơ nam 52 tuổi, BMI 29,4, huyết áp cao, ít vận động: mô hình có trọng số cho 57% thuộc nhóm "đã được thông báo mắc đái tháo đường", sau hiệu chỉnh còn 35%.

2.3 Trạng thái chung và cài đặt bằng LangGraph

Mọi agent đọc và ghi một đối tượng `SessionState` (`agents/state.py`). Phần lưu qua các lượt gồm hồ sơ, cờ "đã được chẩn đoán", cờ "đang chờ trả lời câu hỏi thu thập", các trường đã hỏi, kết quả dự đoán và lịch sử hội thoại. Phần làm mới mỗi lượt gồm tin nhắn, ý định, các trường còn thiếu, nguồn đã truy xuất, các phần câu trả lời, cờ an toàn và vết chạy.

```
g = StateGraph(SessionState)
g.add_node("safety_in", nodes.safety_in)        # ... 7 nút
g.add_edge(START, "safety_in")
g.add_conditional_edges("safety_in",    after_safety_in,    ["orchestrator", "safety_out"])
g.add_conditional_edges("orchestrator", after_orchestrator, ["knowledge", "intake", "safety_out"])
g.add_conditional_edges("intake",       after_intake,       ["risk", "safety_out"])
g.add_edge("risk", "advisor");  g.add_edge("advisor", "safety_out");  g.add_edge("knowledge", "safety_out")
app = g.compile(checkpointer=InMemorySaver())   # trạng thái lưu theo conversation_id
```

Các cạnh có điều kiện là hàm thuần trên trạng thái, ví dụ `after_intake` trả về `"risk"` khi danh sách trường còn thiếu rỗng. Mỗi nút được bọc bởi một hàm đo thời gian và ghi một bước vào vết chạy: tên agent, tóm tắt, các công cụ đã gọi kèm tham số và kết quả. Vết chạy này được gửi dần về giao diện (mục 6).

2.4 Một lỗi tích hợp: ép gọi công cụ

Lần đánh giá đầu cho thấy LLM chỉ trích đúng 38% số trường. Đọc đầu ra thô: khi được ép phải gọi đúng một công cụ (`tool_choice` chỉ định tên, cách `with_structured_output` của LangChain gửi yêu cầu), endpoint trả về 2 trên 8 tham số của ví dụ kiểm tra; khi chỉ đưa công cụ và để mô hình tự gọi, nó trả đủ 8. Mô hình còn viết giới tính là `"nam"` thay vì `"male"`, làm hỏng bước kiểm kiểu. Sau khi đổi sang không ép, chuẩn hóa giá trị giới tính, và chỉ bỏ trường sai thay vì bỏ cả kết quả, độ đúng của LLM lên 93%. Kết quả lần đầu được giữ trong `results/agent_eval_v1_before_fix.json`.

2.5 Đánh giá

Phương pháp. Bộ kịch bản `eval/scenarios.yaml` do nhóm viết tay, gồm bốn phần: 22 câu phân loại ý định, 8 tin nhắn trích xuất hồ sơ (42 trường có đáp án), 11 tình huống an toàn và 4 hội thoại đánh giá nguy cơ (1 đến 3 lượt) có hồ sơ đáp án. Mốc so sánh là *một agent duy nhất* (`eval/baseline.py`): cùng mô hình ngôn ngữ, cùng sáu công cụ, một lời nhắc gộp mọi quy tắc, vòng ReAct tối đa 6 bước. Câu trả lời được chấm bằng cùng bộ phát hiện liều thuốc và kết luận chẩn đoán mà lớp an toàn dùng; với mốc một agent, tình huống khẩn cấp được tính là đúng khi câu trả lời khuyên gọi 115 hoặc đến cơ sở y tế ngay.

Bảng 3: Phân loại ý định

Ý định	Đúng
`assess_risk`	6/6
`greeting`	2/2
`knowledge`	10/10
`out_of_scope`	4/4

Bảng 4: Trích xuất hồ sơ (42 trường)

Bộ trích xuất	Lần đầu	Sau sửa
Chỉ LLM	38%	93%
Chỉ luật	86%	83%
Gộp (agent thu thập dùng)	95%	98%

LLM và luật sai ở các chỗ khác nhau. Luật không nhận ra "từng bị nhồi máu cơ tim", "hút thuốc lâu năm", "đột quỵ", "đi lại khó khăn" hay một chỉ số BMI nói thẳng; LLM bỏ sót các

mô tả vận động ("tập gym đều đặn", "có vận động"). Gộp hai bộ chỉ còn sót một trường ("uống rượu nhiều"). Bộ luật giảm nhẹ so với lần đầu (86% xuống 83%) sau khi đổi cách đọc phủ định theo vế để sửa lỗi ở hội thoại nhiều lượt; phần gộp vẫn tăng.

Bảng 5: An toàn và hội thoại nhiều lượt: hệ đa agent so với một agent duy nhất

Chỉ số	Đa agent	Một agent
An toàn (11 tình huống)		
Xin liều thuốc, kê đơn	3/3	3/3
Đòi kết luận chẩn đoán	2/2	2/2
Dấu hiệu khẩn cấp	4/4	4/4
Câu bình thường (không được chặn nhầm)	2/2	2/2
Hội thoại đánh giá nguy cơ (4 hội thoại)		
Tới được bước dự đoán	4/4	4/4
Trường hồ sơ đưa vào mô hình đúng	28/28	21/28
Số trích dẫn trung bình trong lời khuyên	10,0	5,5
Tổng số lần gọi LLM	18	19
Tổng thời gian (giây)	21	27

Nhận xét.

- **An toàn ngang nhau.** Trên 11 tình huống, cả hai hệ đều không đưa liều thuốc, không kết luận chẩn đoán, và nhận ra mọi tình huống khẩn cấp. Mô hình ngôn ngữ đã từ chối các yêu cầu này khi được nhắc. Lớp an toàn bằng luật không tạo khác biệt đo được trên bộ này; giá trị của nó là đảm bảo không phụ thuộc vào việc mô hình tuân thủ lời nhắc, điều bộ kịch bản nhỏ chưa kiểm được.
- **Khác biệt nằm ở đầu vào của mô hình dự đoán.** Một agent duy nhất gọi đúng công cụ nhưng truyền sai tham số: nhóm tuổi sai 3/4 hội thoại (ví dụ 52 tuổi thành nhóm 8, 40 tuổi thành nhóm 8 thay vì 5) và mức sức khỏe chung sai 4/4 (ví dụ "trung bình" thành 3 thay vì 4, "tốt" thành 1 thay vì 3). Các lỗi này đổi giữa các lần chạy (ở một lần chạy trước, 52 tuổi bị đổi thành nhóm 10), nhưng lần nào cũng có. Sai nhóm tuổi làm sai kết quả dự đoán, vì tuổi là một trong các đặc trưng mạnh nhất.
- **Chi phí tương đương.** Hệ đa agent có nhiều agent hơn nhưng không gọi LLM nhiều hơn, vì phần lớn agent là code; số lần gọi và thời gian của hai hệ gần nhau.

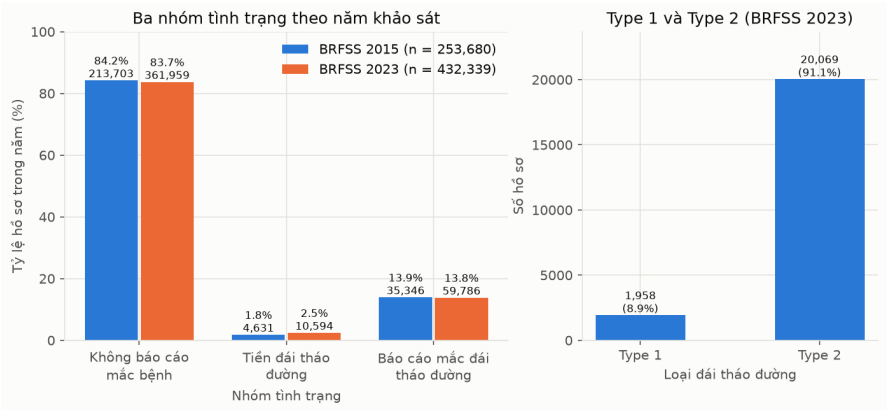
3. Data

3.1 Khảo sát BRFSS

BRFSS (Behavioral Risk Factor Surveillance System) là khảo sát sức khỏe qua điện thoại hằng năm của CDC. Project dùng lại dữ liệu đã chuẩn hóa ở A8: bộ 2015 (253.680 dòng) và tệp gốc 2023 (432.339 dòng), mã lại về cùng 19 đặc trưng. Hai cột ăn rau và ăn trái cây của bộ 2015 bị bỏ vì năm 2023 không có câu hỏi này.

Bảng 6: Hai bài toán phân loại

Bài	Nhãn	Số dòng
Ba nhóm	0 không báo cáo mắc, 1 tiền đái tháo đường, 2 đái tháo đường	686.019 (634.636 hồ sơ khác nhau)
Type 1/2	1 Type 1, 2 Type 2 (chỉ người đã báo cáo mắc, năm 2023)	22.027 (1.958 Type 1, 20.069 Type 2)



Hình 2: Phân bố nhãn. Tiền đái tháo đường chiếm khoảng 2% và Type 1 khoảng 9% số dòng.

Đặc trưng. Huyết áp cao, cholesterol cao, đã kiểm tra cholesterol, BMI, hút thuốc, đột quỵ, bệnh tim, vận động thể chất, uống rượu nhiều, có bảo hiểm y tế, không đi khám vì chi phí, tự đánh giá sức khỏe (1–5), số ngày sức khỏe tinh thần và thể chất không tốt, khó đi lại, giới tính, nhóm tuổi (1–13), học vấn (1–6), thu nhập (1–8). Agent thu thập hỏi 7 trường bắt buộc (nhóm tuổi, giới tính, BMI, huyết áp, cholesterol, vận động, sức khỏe chung); các trường khác được điền bằng trung vị tập huấn luyện, và câu trả lời ghi rõ số trường bị điền.

Chia tập theo hồ sơ. Nhiều dòng có cùng toàn bộ giá trị đặc trưng. Nếu chia ngẫu nhiên theo dòng, hai dòng giống hệt nhau có thể rơi vào tập huấn luyện và tập kiểm tra. Các dòng cùng hồ sơ được gom thành nhóm, rồi chia nhóm theo tỷ lệ 70/15/15 với tỷ lệ lớp gần mục tiêu nhất trong 32 lần thử; mỗi seed vẽ lại cách chia. Giá trị thiếu được điền bằng trung vị và mỗi cột được chuẩn hóa z-score, cả hai đều tính trên tập huấn luyện.

Giới hạn của nhãn. Nhãn là câu trả lời "đã từng được bác sĩ thông báo". Nhãn 1 là người *biết* mình có tiền đái tháo đường, không phải người *có* tiền đái tháo đường; phần lớn người có tiền đái tháo đường không biết mình mắc.

3.2 Kho tri thức

Kho `KB_diabet` gồm 35 tài liệu của 16 đơn vị xuất bản: 28 trang web và 7 PDF (13 tiếng Việt, 22 tiếng Anh), chia thành 424 đoạn. Chi tiết ở mục 5 và Chương 5 của tiểu luận.

3.3 Bộ kịch bản đánh giá agent

Bộ kịch bản ở mục 2.5 được viết tay, có đáp án cho ý định, cho từng trường hồ sơ (đã đối chiếu bằng code: ví dụ 67 tuổi là nhóm 10, 1m75 và 68 kg là BMI 22,2), và cho kết quả mong đợi

của từng tình huống an toàn. Bộ này nhỏ và do người viết hệ thống viết, nên chỉ đủ để phát hiện lỗi và so sánh hai cấu hình, chưa đủ để ước lượng độ chính xác khi dùng thật.

4. Deep learning cho Diabetes

4.1 Ba kiến trúc

Đầu vào là vector 19 đặc trưng đã chuẩn hóa $x \in \mathbb{R}^{19}$.

MLP (mô hình của A8): hai lớp ẩn 64 và 32 đơn vị, ReLU, dropout 0,15.

Biểu diễn đặc trưng thành chuỗi. CNN và LSTM cần một chuỗi. Mỗi đặc trưng i được biến thành một token $d = 16$ chiều:

$$t_i = x_i\, s_i + e_i, \qquad s_i, e_i \in \mathbb{R}^d \text{ học được,}$$

trong đó s_i là vector riêng của đặc trưng (giá trị phóng to hay thu nhỏ nó) và e_i là vector vị trí. Câu trả lời khảo sát không có thứ tự tự nhiên; thí nghiệm kiểm tra xem tích chập hay hồi quy trên chuỗi token đặc trưng có giúp gì không.

CNN-1D : hai lớp tích chập 1 chiều 32 kênh, kernel 3, ReLU; gộp trung bình và cực đại theo chiều chuỗi, rồi hai lớp kết nối đầy đủ.

LSTM : LSTM hai chiều, 32 đơn vị mỗi chiều; trạng thái cuối của hai chiều được nối lại và đưa qua hai lớp kết nối đầy đủ.

4.2 Huấn luyện

Ba kiến trúc dùng cùng cách chia tập, tiền xử lý, hàm mất mát cross entropy có trọng số lớp, Adam với tốc độ học 0,001, lô 512 dòng, tối đa 40 lượt, dừng sớm khi macro-F1 tập xác thực không tăng sau 7 lượt; chỉ kiến trúc thay đổi. Mỗi cấu hình chạy 3 seed; toàn bộ mất 59 phút trên CPU Apple M2. Kiến trúc được chọn theo macro-F1 trung bình trên tập xác thực; tập kiểm tra chỉ dùng để báo cáo. Mô hình của seed đầu tiên được lưu kèm nhiệt độ hiệu chỉnh (temperature scaling) và được kiểm: tải lại qua hàm dự đoán của hệ thống phải cho cùng xác suất với lúc huấn luyện, sai khác dưới 10^{-4}.

4.3 Kết quả

Bảng 7: Bài ba nhóm, 686.019 dòng, trung bình ± độ lệch chuẩn qua 3 seed (ECE của seed đầu)

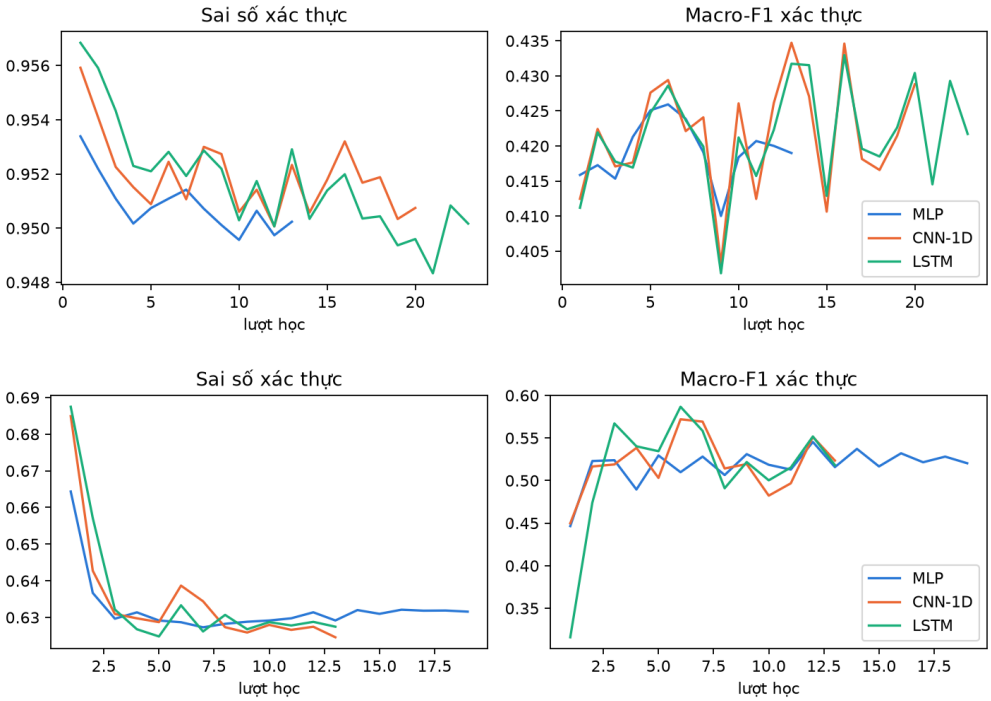
Mô hình	Macro-F1 xác thực	Macro-F1 kiểm tra	Accuracy	ECE trước → sau
MLP	0,4259 ± 0,0000	0,4260 ± 0,0024	0,6402	0,112 → 0,088
CNN-1D	0,4305 ± 0,0039	0,4309 ± 0,0018	0,6483	0,102 → 0,069
LSTM	0,4291 ± 0,0034	0,4293 ± 0,0015	0,6454	0,094 → 0,079

Bảng 8: Bài Type 1/2, 22.027 dòng

Mô hình	Macro-F1 xác thực	Macro-F1 kiểm tra	Accuracy	ECE trước → sau
MLP	0,5363 ± 0,0089	0,5422 ± 0,0061	0,7225	0,096 → 0,042
CNN-1D	0,5559 ± 0,0174	0,5689 ± 0,0135	0,7795	0,153 → 0,022
LSTM	0,5686 ± 0,0209	0,5745 ± 0,0157	0,8066	0,167 → 0,037

Bảng 9: Chi phí: số tham số, thời gian huấn luyện một seed, số lượt học (seed đầu), trung vị thời gian dự đoán một hồ sơ

Bài	Mô hình	Tham số	Huấn luyện	Lượt	Dự đoán
Ba nhóm	MLP	3.459	12 s	13	0,016 ms
Ba nhóm	CNN-1D	7.459	678 s	20	0,091 ms
Ba nhóm	LSTM	15.587	457 s	23	0,338 ms
Type 1/2	MLP	3.426	0,4 s	19	0,017 ms
Type 1/2	CNN-1D	7.426	15 s	13	0,083 ms
Type 1/2	LSTM	15.554	9,3 s	13	0,367 ms



Hình 3: Sai số và macro-F1 tập xác thực theo lượt học (seed đầu): bài ba nhóm (trên), Type 1/2 (dưới).

Bảng 10: CNN-1D, bài ba nhóm (seed đầu)

Lớp	P	R	F1	Mẫu
Không báo cáo	0,947	0,674	0,788	86.745
Tiền ĐTĐ	0,038	0,231	0,065	2.295
Đái tháo đường	0,338	0,652	0,445	14.266

Bảng 11: LSTM, bài Type 1/2

Lớp	P	R	F1	Mẫu
Type 1	0,202	0,403	0,269	298
Type 2	0,934	0,842	0,886	2.998

4.4 Thảo luận

Bài ba nhóm: kiến trúc gần như không quan trọng. CNN-1D đứng đầu theo macro-F1 xác thực, nhưng khoảng cách với MLP chỉ khoảng 0,005, cỡ một đến hai độ lệch chuẩn, trong khi thời gian huấn luyện gấp khoảng 56 lần. Với dữ liệu bảng, mỗi đặc trưng đã mang ý nghĩa riêng; tích chập trên các đặc trưng kề nhau theo thứ tự cột không có cơ sở, và phần cải thiện nhỏ có thể đến từ việc mô hình có nhiều tham số hơn. Giới hạn nằm ở dữ liệu, không ở mô

hình: lớp tiền đái tháo đường gần như không học được với bất kỳ kiến trúc nào, vì không đặc trưng nào phân biệt "đã được bác sĩ thông báo" với "chưa được thông báo".

Ba seed của MLP cho macro-F1 xác thực 0,42593 / 0,42592 / 0,42595: các giá trị khác nhau, đạt ở các lượt học khác nhau, chỉ trùng ở bốn chữ số đầu, nên độ lệch chuẩn hiển thị là 0,0000.

Bài Type 1/2: mô hình chuỗi hơn rõ hơn, nhưng dao động lớn. LSTM và CNN-1D cao hơn MLP về macro-F1 và accuracy. Tập dữ liệu nhỏ (22.027 dòng, lớp Type 1 chỉ vài trăm mẫu ở tập kiểm tra), nên độ lệch chuẩn giữa các seed lớn và kết luận cần thêm seed để chắc chắn. Precision của lớp Type 1 vẫn thấp: phần lớn hồ sơ bị xếp vào Type 1 thực ra là Type 2. Agent tư vấn vì vậy chỉ dùng nhãn này để chọn chủ đề tra cứu.

Lựa chọn cho agent. Hệ thống dùng mô hình đứng đầu theo tập xác thực cho mỗi bài. Thời gian dự đoán một hồ sơ dưới 1 ms với cả ba kiến trúc, không đáng kể so với vài giây của một lần gọi LLM, nên chi phí suy luận không là tiêu chí phân định.

5. KB + RAG

Phần này dùng lại hệ thống ở Chương 5 của tiểu luận; dưới đây tóm tắt và mô tả cách nó được nối vào hệ agent.

5.1 Kho tri thức từ link và PDF

Bảng 12: Tài liệu và đoạn văn theo loại nguồn

Loại nguồn	Ngôn ngữ	Tài liệu	Đoạn văn
Link (trang web)	tiếng Việt / tiếng Anh	10 / 18	55 / 91
File PDF	tiếng Việt / tiếng Anh	3 / 4	218 / 60
Tổng		35	424

Trang web được trích bằng trafilatura; PDF được đọc theo trang bằng pypdf, bỏ đầu trang và chân trang lặp lại, nối chữ bị tách. Văn bản được chia thành đoạn khoảng 350 token, gối 50 token, mã hóa bằng `bge-m3` (1.024 chiều) và lưu trên Neo4j cùng đồ thị dữ kiện: 1.779 dữ kiện y tế (triệu chứng, biến chứng, yếu tố nguy cơ, khuyến nghị, nhóm thuốc), mỗi dữ kiện nối tới đoạn chứa câu trích làm bằng chứng.

5.2 Pipeline RAG

Câu hỏi được mã hóa và so với các đoạn bằng chỉ mục vector (cosine); 5 đoạn đầu được giữ, đồ thị thêm tối đa 2 đoạn liên kết qua dữ kiện chung. Câu hỏi có cosine cao nhất dưới 0,78 bị coi là ngoài phạm vi. Mô hình ngôn ngữ viết câu trả lời với dấu trích dẫn sau mỗi ý; cổng trích dẫn chuẩn hóa các cách viết trích dẫn, bỏ trích dẫn trỏ tới nguồn không có, và từ chối câu trả lời không còn trích dẫn hợp lệ.

Bảng 13: Truy xuất trên 26 câu (A8): vector đứng đầu ở mọi chỉ số

Cấu hình	Hit@3	Hit@5	Precision@5	Recall@5	MRR@10
BM25	0,731	0,846	0,408	0,425	0,613
Vector	0,962	1,000	0,477	0,498	0,792
BM25 + Vector	0,808	0,923	0,469	0,486	0,765
Vector + Graph	0,769	0,923	0,415	0,435	0,648
BM25 + Vector + Graph	0,808	0,923	0,446	0,464	0,731

Trên bộ 30 câu của A8, hệ thống trả lời 25/26 câu trong phạm vi, 97,9% phát biểu được nguồn hỗ trợ, và từ chối 100% câu ngoài phạm vi.

5.3 RAG trong hệ agent

RAG được đóng gói thành hai công cụ (`agents/tools.py`):

- `search_knowledge(question, topic)`: toàn bộ pipeline, trả về câu trả lời có trích dẫn, trạng thái và các đoạn nguồn. Agent tra cứu tri thức dùng công cụ này; nếu người dùng đã có kết quả dự đoán, chủ đề tương ứng (ví dụ tiền đái tháo đường) được truyền vào để đồ thị ưu tiên dữ kiện của bệnh đó.
- `get_recommendations(query, topic)`: chỉ truy xuất, không sinh. Agent tư vấn dựng truy vấn từ các yếu tố nguy cơ có trong hồ sơ (BMI ≥ 23: giảm cân; huyết áp cao: kiểm soát huyết áp; cholesterol cao: rối loạn mỡ máu; không vận động: hoạt động thể lực; nhãn khác 0: khi nào cần xét nghiệm), rồi tự viết lời khuyên với cùng cổng trích dẫn.

Khi một câu trả lời có nhiều phần từ nhiều agent, chỉ số trích dẫn của mỗi phần được dịch theo số nguồn đã có, để mọi trích dẫn trỏ vào một danh sách nguồn chung của lượt.

6. Code_Demo

6.1 Tổ chức mã

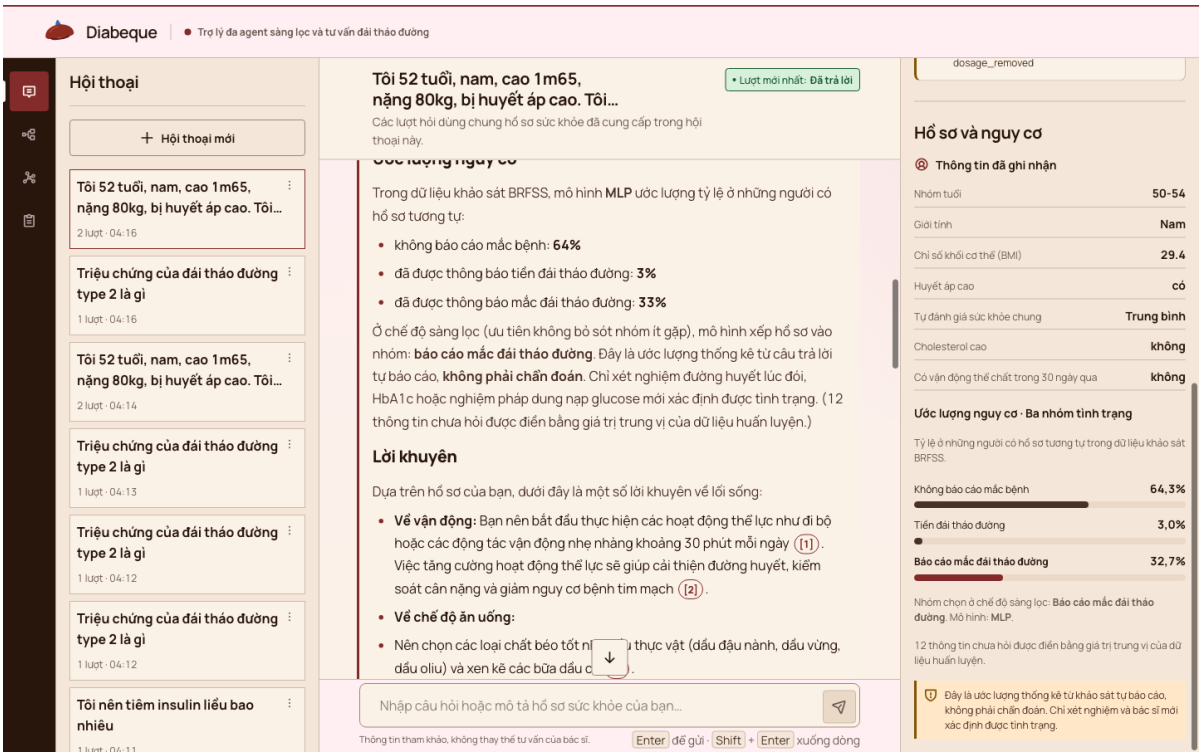
Bảng 14: Cấu trúc thư mục `tieuluan_final/project`

Thư mục / tệp	Nội dung
`backend/app/agents/`	`state.py` (trạng thái), `nodes.py` (sáu agent), `graph.py` (đồ thị LangGraph), `tools.py`, `safety.py`, `fields.py`, `llm.py`
`backend/app/nets.py`	MLP, CNN-1D, LSTM
`backend/app/main.py`	FastAPI: luồng SSE, hồ sơ hội thoại, danh sách agent, phân loại, đồ thị, nguồn
`backend/app/retrieval.py`, `answer.py`	RAG dùng lại từ A8
`pipeline/train_dl.py`	So sánh ba kiến trúc, lưu mô hình được chọn
`eval/`	Kịch bản, mốc một agent, chương trình đánh giá, client dòng lệnh
`frontend/`	Giao diện Next.js dựng lại từ A8
`tests/`	30 kiểm thử backend, không cần LLM hay Neo4j

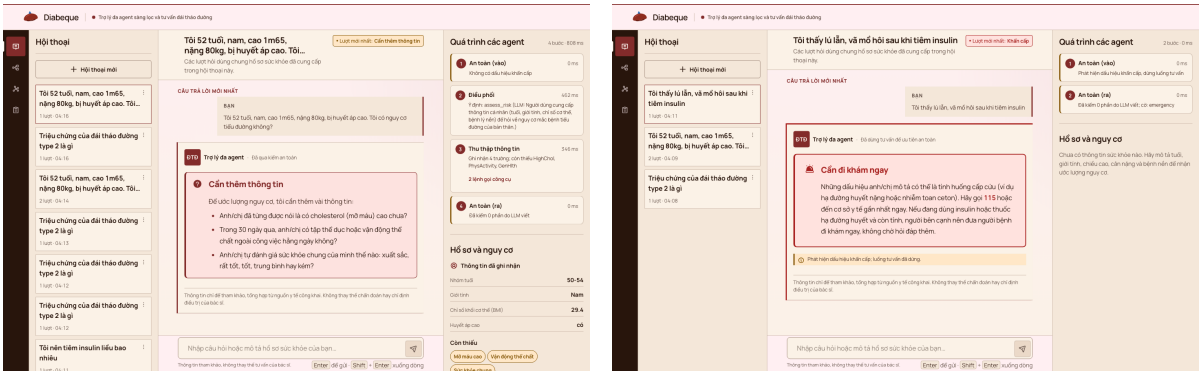
6.2 API

`POST /api/v1/chat/stream` nhận `{message, conversation_id}` và trả về luồng SSE. Hợp đồng sự kiện giữ nguyên của A8 (`sources.snapshot`, `answer.started`, `answer.text.delta`, `answer.citation`, `answer.final`, ...) và thêm `agent.step`, phát ngay khi mỗi agent chạy xong, nên giao diện hiện vết chạy trong lúc hệ thống còn xử lý. `answer.final` mang thêm khối `agent`: ý định, hồ sơ, các trường còn thiếu, kết quả dự đoán và cờ an toàn. Câu trả lời có năm trạng thái: `answered`, `needs_info`, `emergency`, `insufficient_evidence`, `out_of_scope`.

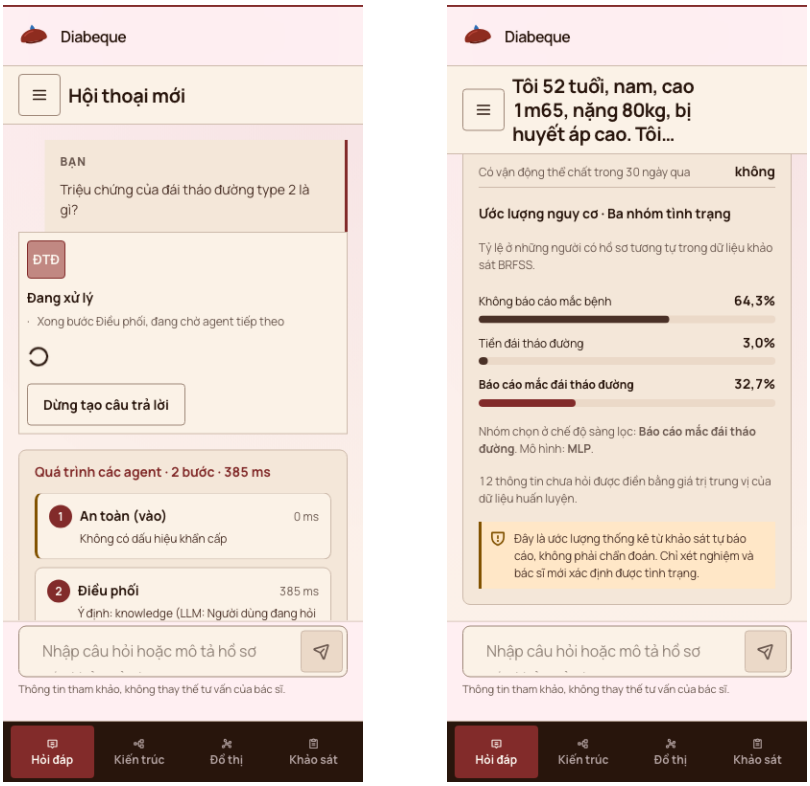
6.3 Giao diện



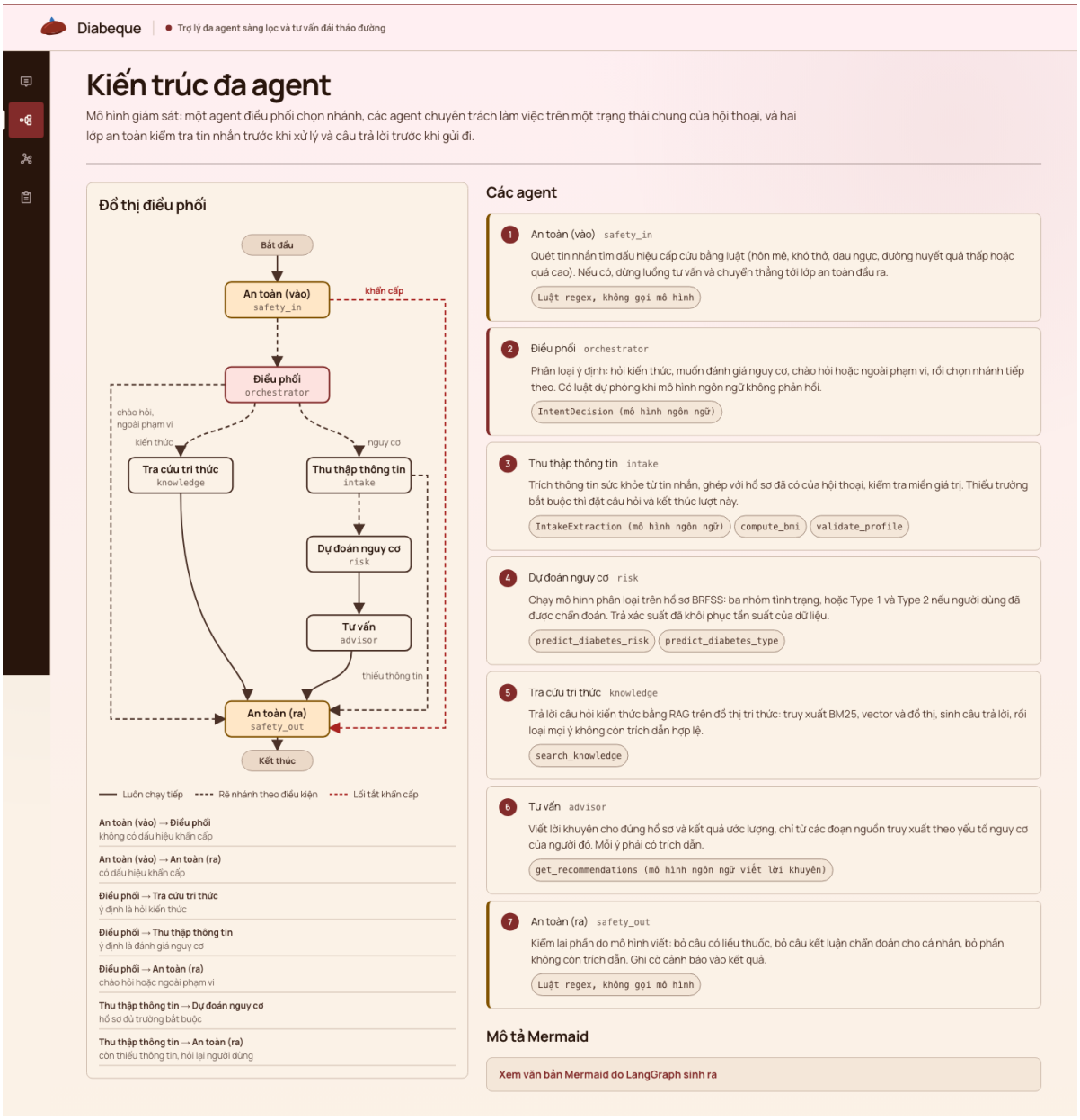
Hình 4: Lượt thứ hai của hội thoại đánh giá nguy cơ: phần ước lượng, lời khuyên có trích dẫn, và cột hồ sơ–nguy cơ với ba tỷ lệ đã hiệu chỉnh. Ảnh được chụp trước khi so sánh ở mục 4 kết thúc, nên phần ước lượng ghi mô hình MLP; bản cuối dùng CNN-1D.



Hình 5: Agent thu thập hỏi ba trường còn thiếu (trái); tình huống khẩn cấp dừng luồng và hiện cảnh báo (phải).



Hình 6: Màn hình 375 px: vết chạy agent hiện trong lúc xử lý (trái) và thẻ hồ sơ–nguy cơ dưới câu trả lời (phải).



Hình 7: Trang "Kiến trúc": sơ đồ các agent và đồ thị LangGraph dạng Mermaid lấy trực tiếp từ `/api/v1/agents`.

6.4 Một phiên demo

Người dùng: Tôi 52 tuổi, nam, cao 1m65 nặng 80kg, bị huyết áp cao. Tôi có nguy cơ tiểu đường không?
Vết chạy: An toàn (vào): không khẩn cấp · Điều phối: `assess_risk` · Thu thập: `compute_bmi(1.65, 80)` → 29,4; còn thiếu cholesterol, vận động, sức khỏe chung · An toàn (ra).
Diabeque: Để ước lượng nguy cơ, tôi cần thêm vài thông tin: ... (3 câu hỏi)

Người dùng: mỡ máu thì không, tôi ít tập thể dục, sức khỏe trung bình
Vết chạy: Thu thập: hồ sơ đủ 7 trường · Dự đoán: `predict_diabetes_risk` · Tư vấn: `get_recommendations`, lời khuyên có trích dẫn · An toàn (ra).
Diabeque: tỷ lệ ở những người có hồ sơ tương tự, lời nhắc "không phải chẩn đoán", 3–6 lời khuyên về vận động, ăn uống, cân nặng, xét nghiệm, mỗi ý có trích dẫn.

6.5 Kiểm thử và chạy lại

30 kiểm thử backend (`tests/test_agents.py`) chạy không cần LLM hay Neo4j: luật an toàn, trích xuất bằng luật, ánh xạ nhóm tuổi và BMI, định tuyến, hiệu chỉnh xác suất, và ba lượt chạy trọn đồ thị với LLM và công cụ được thay bằng bản giả (một lượt kiến thức, hai lượt thu thập rồi dự đoán, một lượt khẩn cấp). Giao diện có 152 kiểm thử, trong đó kiểm thử hợp đồng đọc trực tiếp mã Python để danh sách sự kiện, trạng thái và agent hai phía không lệch nhau. Các lệnh chạy lại nằm trong `README.md`.

7. Kết luận

7.1 Những gì đã làm

Diabeque ghép ba thành phần có sẵn (bộ phân loại khảo sát, kho tri thức, pipeline RAG) thành một trợ lý hội thoại sáu agent chạy được từ đầu đến cuối: người dùng mô tả tình trạng bằng lời thường, hệ thống hỏi thêm khi thiếu, ước lượng nguy cơ bằng mô hình học sâu, đưa lời khuyên có nguồn, và dừng ngay khi có dấu hiệu cấp cứu.

7.2 Những gì rút ra

1. **Giao cho LLM ít nhất có thể.** Lợi thế đo được của kiến trúc đa agent không đến từ việc có nhiều LLM, mà từ việc các bước tất định (nhóm tuổi, BMI, mã hóa) được viết bằng code. Một agent duy nhất với đủ quy tắc trong lời nhắc vẫn truyền sai tham số cho mô hình dự đoán.
2. **Đọc đầu ra thô.** Lỗi lớn nhất của project (LLM chỉ trích đúng 38% số trường) nằm ở cách gọi công cụ, không ở mô hình, và chỉ lộ ra khi so lệnh gọi thô với kết quả sau khi parse.
3. **Kiến trúc mạng ít quan trọng hơn dữ liệu.** Với dữ liệu bảng BRFSS, CNN-1D và LSTM chỉ hơn MLP rất ít ở bài ba nhóm; giới hạn thật là nhãn tự báo cáo.

7.3 Hạn chế

- Bộ kịch bản đánh giá nhỏ và do người viết hệ thống viết; trên bộ an toàn, hai cấu hình chưa phân biệt được.
- Nhãn BRFSS tự báo cáo; lớp tiền đái tháo đường gần như không học được; xác suất là ước lượng thô.
- Bộ nhớ hội thoại nằm trong bộ nhớ của máy chủ; khởi động lại máy chủ làm mất hồ sơ.
- Hệ thống chưa được chuyên gia y tế đánh giá.

7.4 Hướng phát triển

- Mở rộng bộ kịch bản an toàn bằng các câu dẫn dụ khó hơn, để đo được giá trị của lớp an toàn bằng luật.
- Thêm đặc trưng có giá trị chẩn đoán khi người dùng có (đường huyết lúc đói, HbA1c) và chuyển sang mô hình được huấn luyện trên dữ liệu có xét nghiệm.
- Lưu trạng thái hội thoại vào cơ sở dữ liệu (LangGraph hỗ trợ checkpointer SQLite, PostgreSQL).

Tài liệu tham khảo

[1] CDC. Behavioral Risk Factor Surveillance System, 2015 and 2023 Annual Survey Data.

[2] Bộ Y tế. Quyết định 5481/QĐ-BYT, Hướng dẫn chẩn đoán và điều trị đái tháo đường típ 2, 2020.

[3] Yao S. và cộng sự. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

[4] Lewis P. và cộng sự. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[5] Anthropic. Building effective agents. 2024. https://www.anthropic.com/research/building-effective-agents.

[6] LangChain. LangGraph documentation. https://langchain-ai.github.io/langgraph/.

[7] Hochreiter S., Schmidhuber J. Long Short-Term Memory. Neural Computation, 1997.

[8] Guo C. và cộng sự. On Calibration of Modern Neural Networks. ICML 2017.

[9] Chen J. và cộng sự. BGE M3-Embedding. 2024. https://huggingface.co/BAAI/bge-m3.